

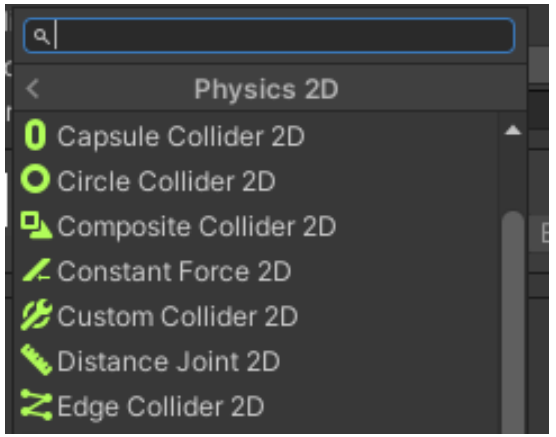
2D Materials
Camera Follow+Zoom
Particle System

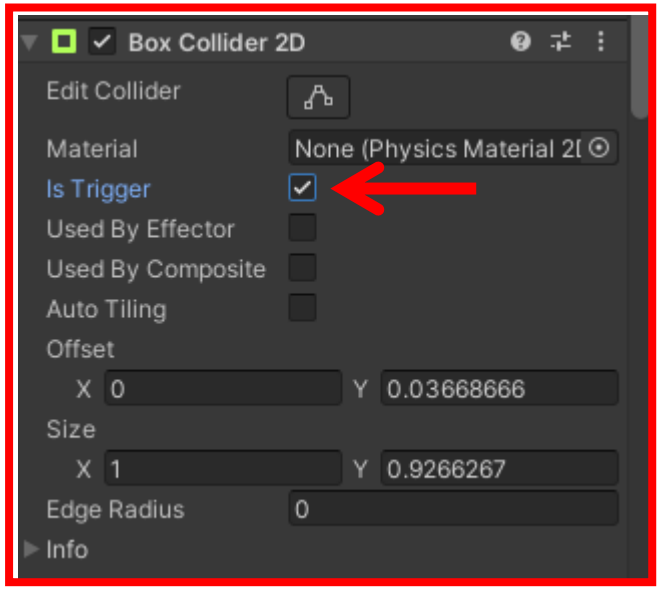
Some Important Rigidbody2D properties

- **Mass:** the weight of an object/character
- **Kinematic:** gravity doesn't affect the object
- **Gravity:** you can make it behave exactly like in the real world, or you could make a world with little to no gravity. Gravity affects how things move, how high character can jump, and how an object reacts to a force against it
- **Linear Drag:** air resistance when moving along x/y axes
- **Angular Drag:** air resistance when rotating along x/y axes
- **Fixed Angle:** can the object spin when hit by something else?

Some Important Collider2D properties

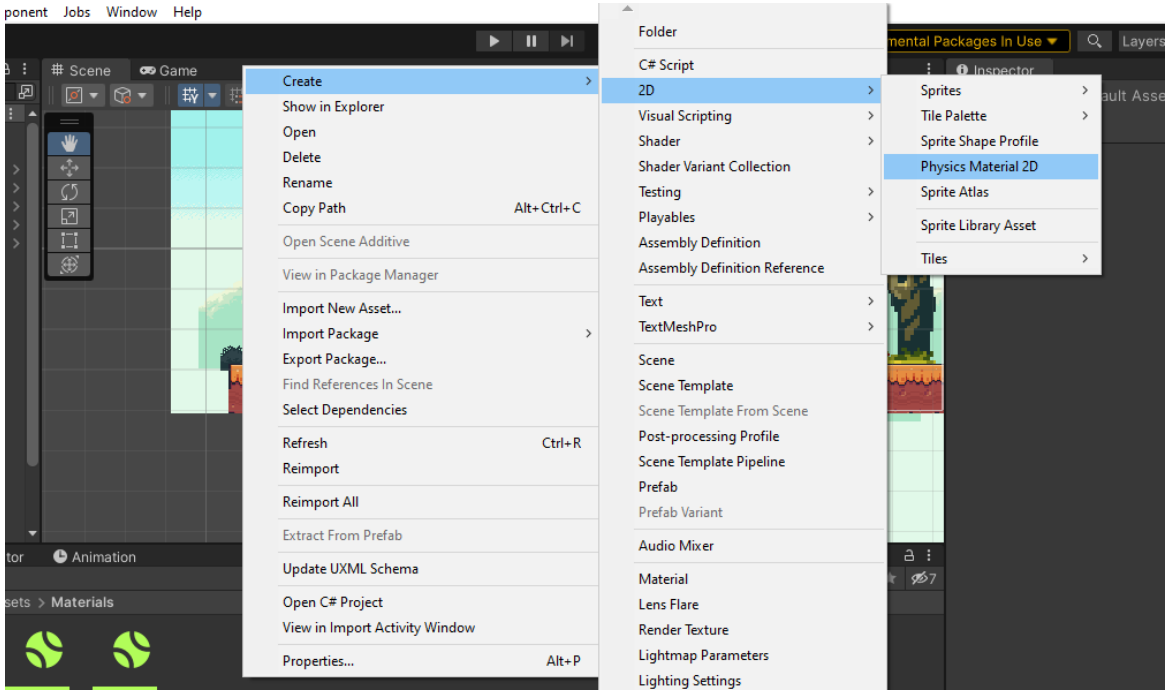
- Colliders can be box-shaped, circular, edge, or polygonal. Properties vary from one type to another
  - **Material:** property to assign a type of material to affect objects that collide with it
  - **Radius:** how big or small the circle collider is around the game object
  - **Size:** how big or small the box collider is around the game object
  - **Offset:** how far away the collider is from the game object itself
  - **Is Trigger:** If checked, a GameObject can trigger an event when it collides with another GameObject. This is useful for setting up events you want to happen when your player comes in contact with them
    - Example: when the player runs into an enemy, it dies

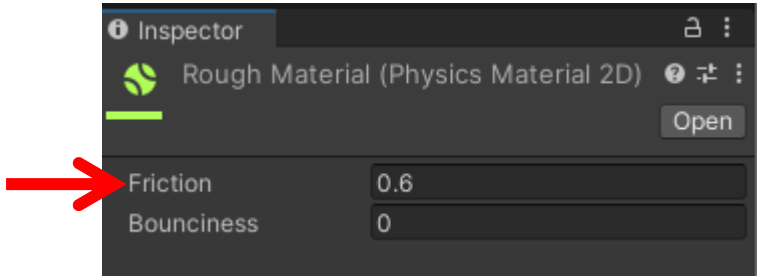




Exercise #10 - Creating Friction

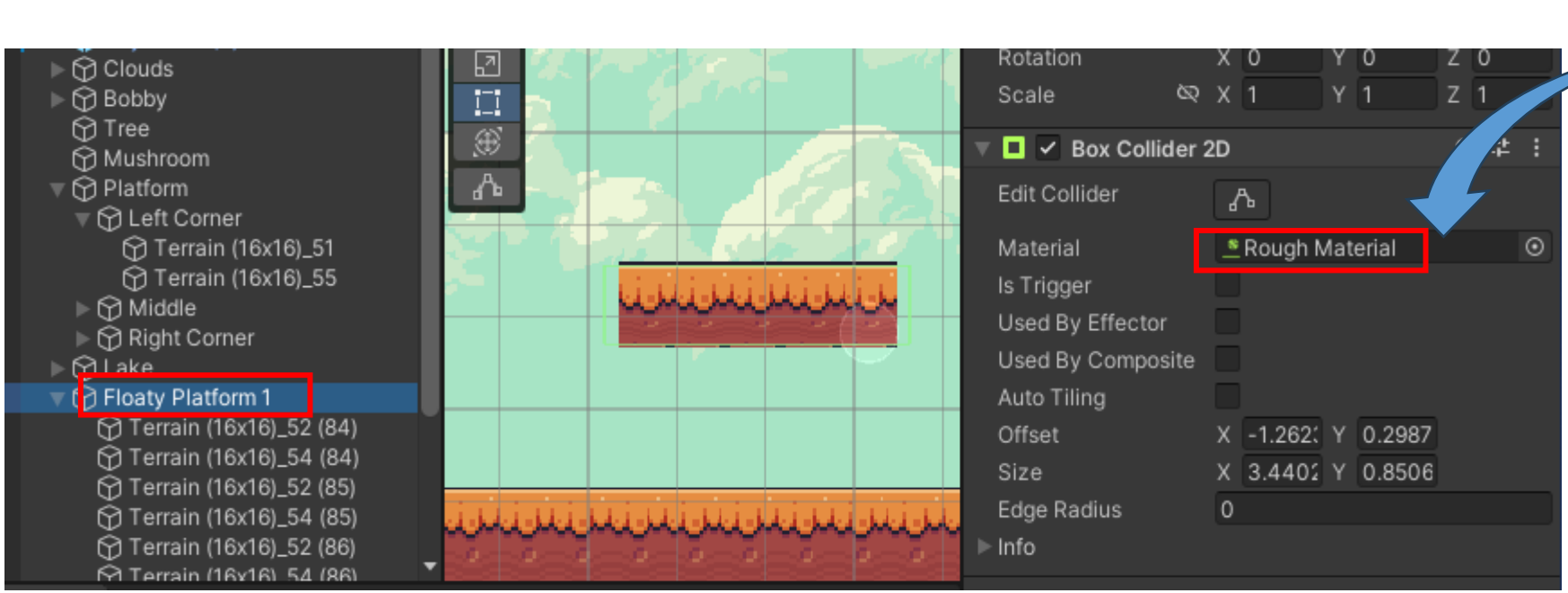
- If your ground/platform is much too slippery, you can make it rougher by controlling the Friction
- Right-click on Assets, and Choose Create > 2D > Physics2D
- Name the new material and go to its Inspector view
- Change the value of friction to whatever suits you
    - Zero is very slippery. One and above is rough.

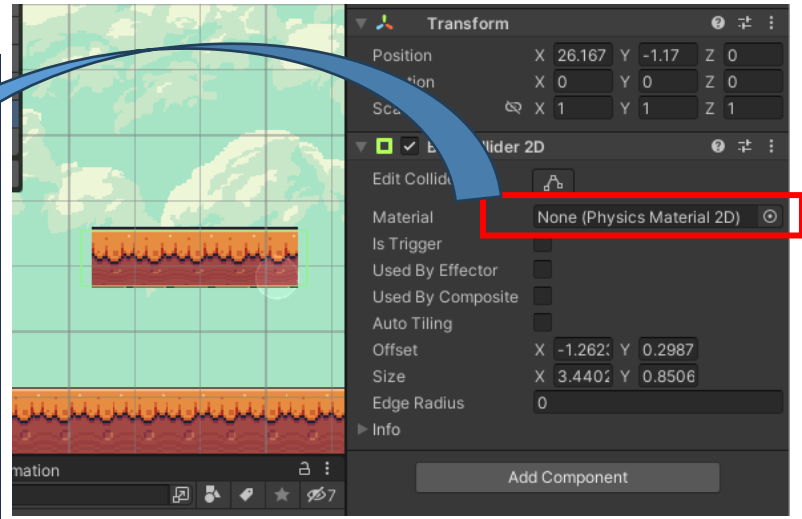




Exercise #10 - Creating Friction (Cont.)

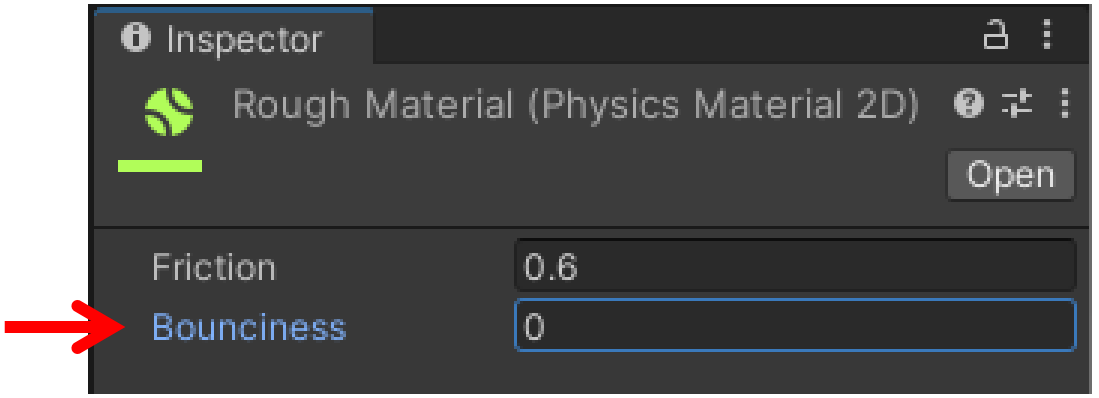
- Now you must attach the material to your ground for the friction to work
  - Make sure your ground already has a box collider component attached!
- **Select the material from your Assets folder**, and drag it to the Material property inside the Box Collider component the Inspector window and drop it
- Now the ground will become as rough or as smooth as you need it to be





Exercise #11 - Creating Bounciness

- Say you have a platform that the player can bounce on like a trampoline
- The procedure is the same as in the previous friction exercise
- **Right-click on Assets, and Choose Physics2D Material**
- Name the new material and go to its Inspector view
- You will notice that the material has 2 fields , we have already discussed the friction. Now the second field is Bounciness. Select the second field.
- Change the value of bounciness
  - Zero indicates no bounce at all
  - Give it a value you desire between 0 and 1



Exercise #11 - Creating Bounciness (Cont.)

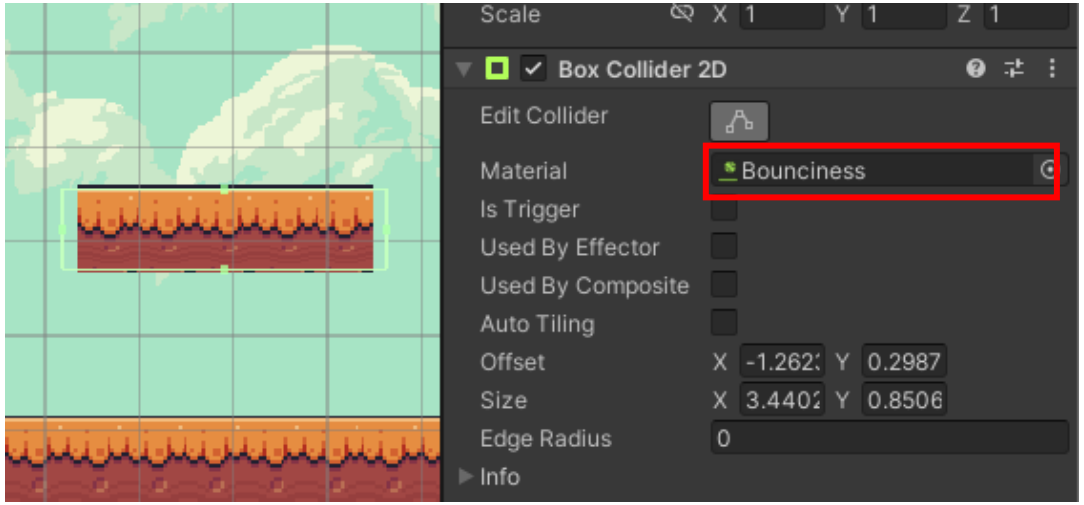
- Make sure the platform has a collider2D component attached
- **Incase you want to make a bouncy effect** , Drag the bouncy material from the Assets folder all the way to the Material property inside the Collider2D component the Inspector window and drop it





- Now the player will bounce upon

landing on the platform



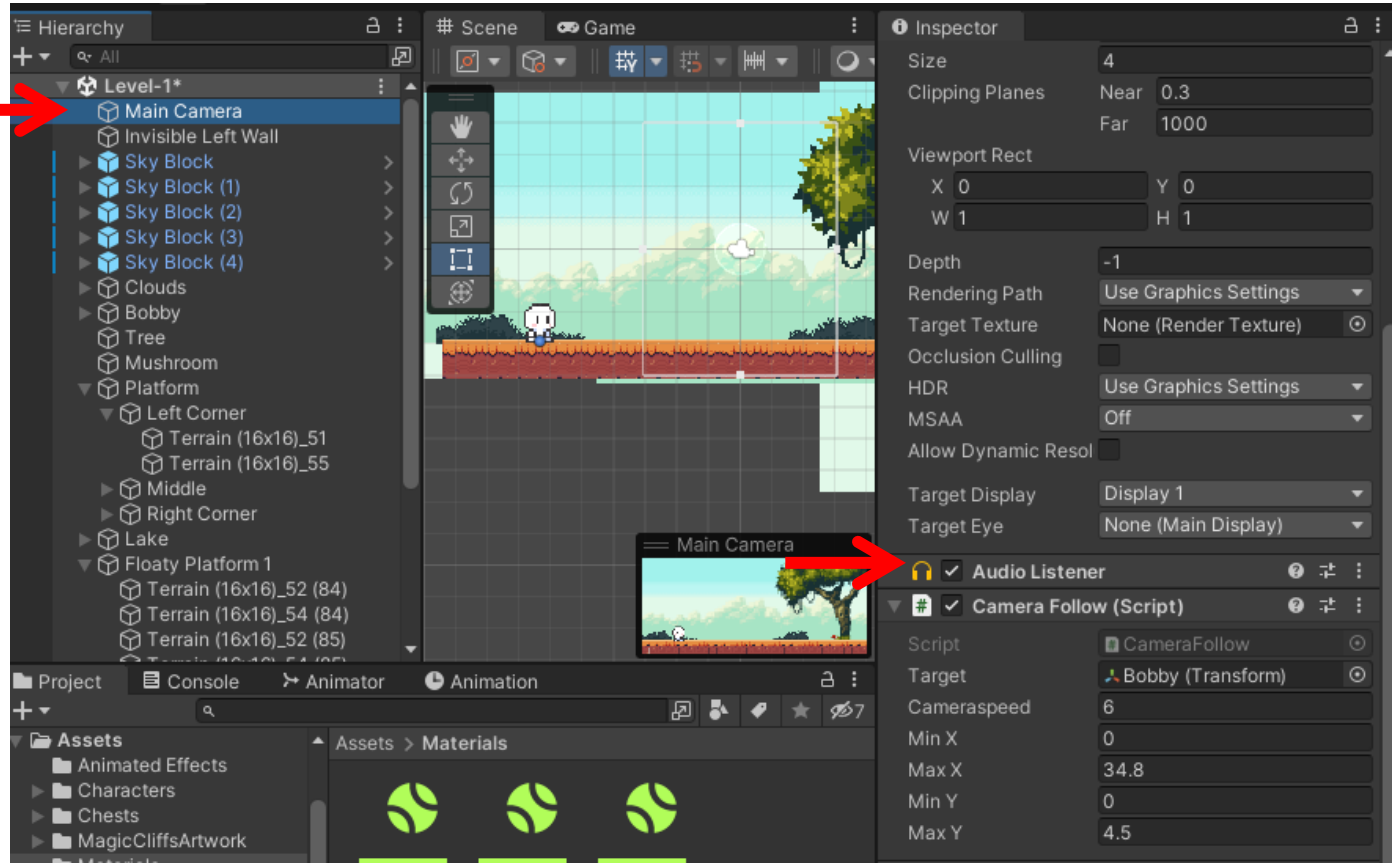
Exercise #12 - Camera Follow

- To make the camera follow your player throughout the level, you must select the camera ,create a script and attach it to your Main Camera.

- Create **New Component > New Script > Name it CameraFollow** or any suitable name.

- Attach CameraFollow to Main Camera.

- Open script in VS Code.



Exercise #12 - Camera Follow (Cont.)

- Create a public variable of type Transform. This will be the object the camera must follow
  - Reminder: any GameObject in your game has a Transform component. It contains the rotation, position and scaling data of that game object

- Create a **public float** and name it CameraSpeed. This value controls how quickly your camera follows the player game object. If your camera's movement is jittery or too slow, you may need to tweak the value

- Control the borders/limits of your camera by creating 4 variables, minX and maxX,minY and maxY

Exercise #12 - Camera Follow (Cont.)

```
public Transform Target;
public float Cameraspeed;

//Determine the minimum and maximum values in both directions where the camera can move. This is to stop it
//from showing game edges or things you don't want to appear to the player
public float minX, maxX;
public float minY, maxY;

void Start()
{
```

Exercise #12 - Camera Follow (Cont.)

- Create a FixedUpdate() function.
    - As long as there's a target to follow (the player character), the camera will always be updating its position to have the same position as that character. Since we're in a 2D game, we will use a variable of type Vector 2. The new camera position is calculated as follows:

```
{
    if(Target!=null)
    {
        Vector2 newCamPosition = Vector2.Lerp(transform.position, Target.position, Time.deltaTime * Cameraspeed);
```

- The Lerp function stands for *Linear Interpolation*. It will help us calculate how fast the camera will move across time, and how it will follow the player game object
- The camera's new position is determined by taking its previous position, target's
- position, and speed multiplied by time that passed since the last frame

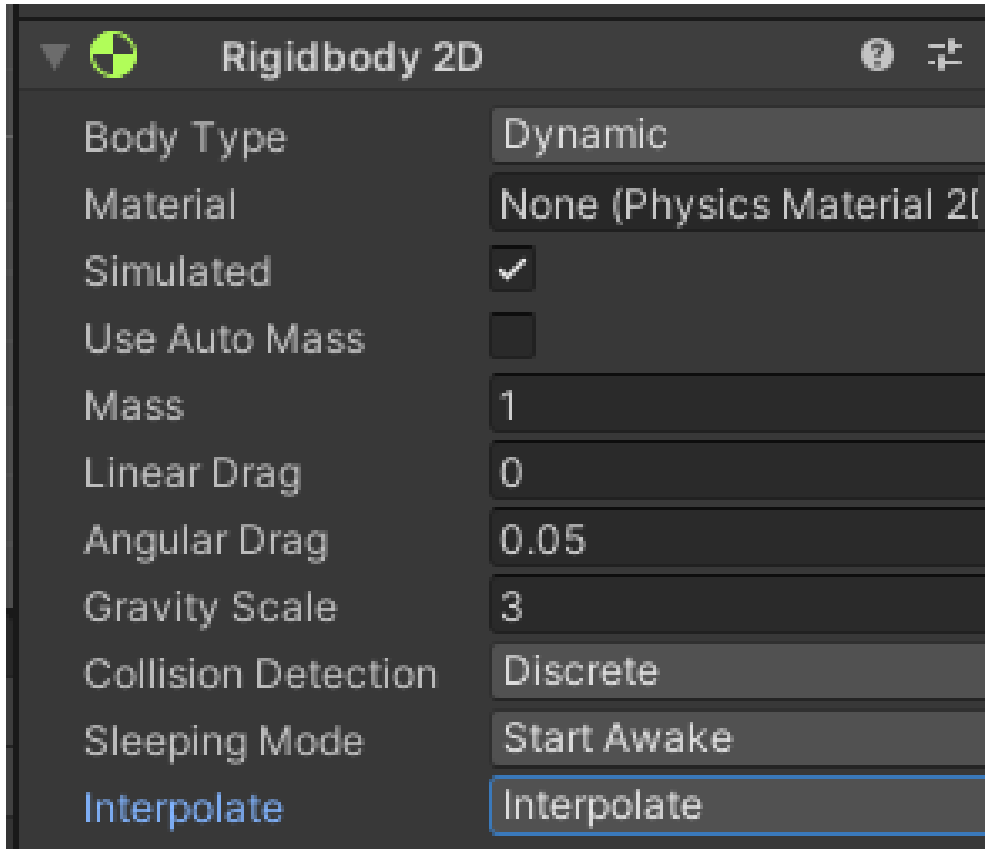
Note on Lerp Function

- One way to explain Linear Interpolation is that it is a mathematical formula (built in unity function) designed to take **3 values/parameters**: (the FROM/BEGINNING point, the TO/ENDING point, and the smoothness by which you arrive to the ending point)

- Example: If you wish to move from point 1 to point 3 on the x-axis, you provide that the FROM value is 1, the TO value is 3, and the smoothness could be 0.1. As you move between points 1 and 3, you'll automatically pass through points 1.1, 1.2, 1.3…2.2,2.3…etc. until you arrive at 3

Exercise #12 - Camera Follow (Cont.)

- Don’t forget to set the Player’s Rigidbody2D interpolate variable from None to ‘Interpolate’.



Exercise #12 - Camera Follow (Cont.)

- Immediately beneath the Lerp function and before the If-condition is over, we add 2 private variables, **ClampX** and **ClampY.** We use them to control the limits of the camera's movement
- Use the Mathf.Clamp function, which needs 3 variables: current Camera position, and the minimum and maximum values on the x and y axis you don't want to exceed

```
        float ClampX = Mathf.Clamp(newCamPosition.x, minX, maxX);
        float ClampY = Mathf.Clamp(newCamPosition.y, minY, maxY);

        transform.position = new Vector3(ClampX, ClampY, -10f);
    }
```

- Finally, update the newest camera position by assigning a Vector3 of values **ClampX** as x, **ClampY** as y, and -10f as z

Exercise #12 - Camera Follow (Cont.)

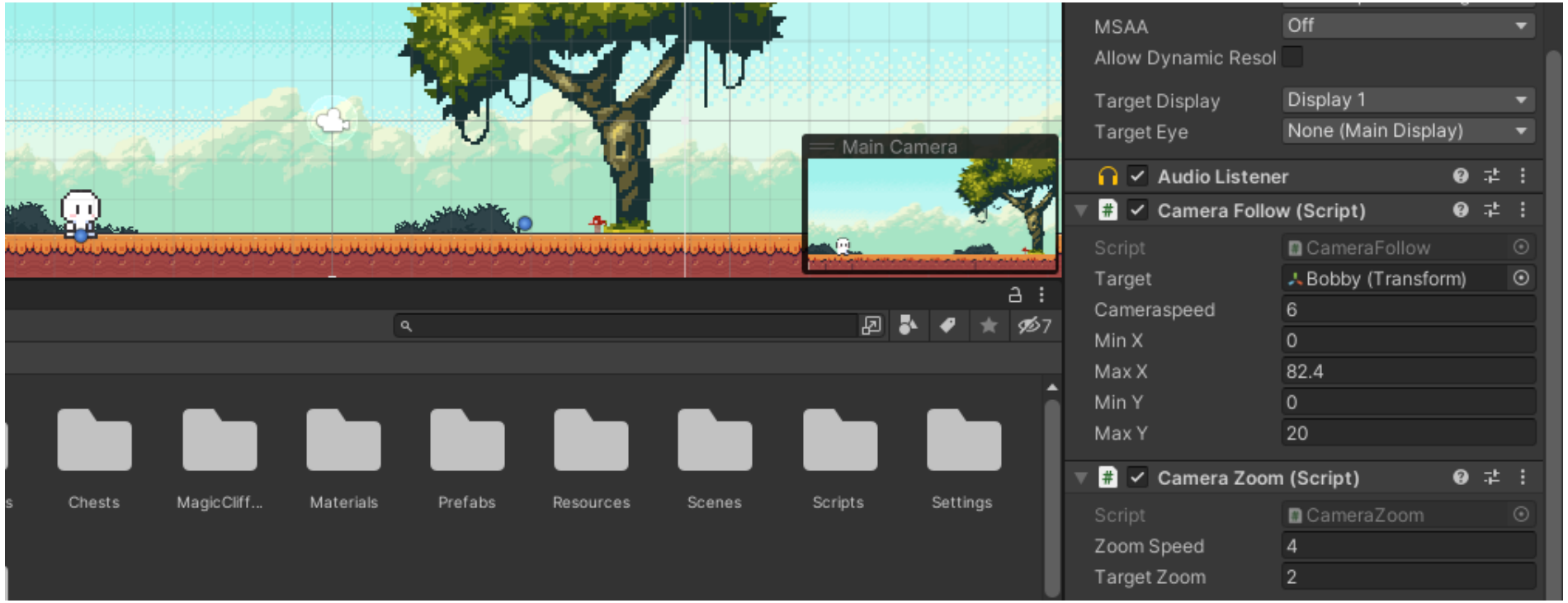
- Go back to Unity
- Give your target variable the player game object by dragging it from the Hierarchy window, and experiment with any speed value. Play around with your camera limits (minX, maxX, etc.) and run your scene



- If your camera moves smoothly, but your player game object doesn't, go to its Rigidbody2D component and turn on Interpolation.
- (Your camera movement may be a bit laggy anyway if the computer's graphics card is limited)

Camera Zoom

- To make your Main Camera zoom in and out, it can be done via Button press, or Mouse Scroll
- To create Zoom via mouse scroll, create a new script and name it CameraZoom



Exercise #13 – Camera Zoom (Mouse Scroll)

- Create the following variables and assign them

```
private Camera Cam;

public float TargetZoom = 3; //The value of Zoom I want by manipulating the Camera Size

private float ScrollData; //Float collected upon Mouse Scrolling

public float ZoomSpeed = 3; //Speed of zooming process in or out


void Start()
{
    Cam = GetComponent<Camera>();

    TargetZoom = Cam.orthographicSize; //The game will begin with TargetZoom being assigned the default given Camera size
}
```

Exercise #13 – Camera Zoom (Mouse Scroll) (Cont.)

- Now in theUpdate()function, use Input.GetAxis(“Mouse ScrollWheel”) to gather float value input and store it in the ScrollData variable

```
// Update is called once per frame
void Update()
{
    //when the Mouse is still, this function returns 0. Forward scrolling returns a +ve value, Bakward scrolling return -ve value.
    ScrollData = Input.GetAxis("Mouse ScrollWheel");
```

- Update the TargetZoom value by subtracting from the ScrollData value

```
//Update the Target Zoom to change from the default to however much I'm scrolling up or down.
TargetZoom = TargetZoom - ScrollData;
```

Exercise #13 – Camera Zoom (Mouse Scroll) (Cont.)

- To make sure your zooming in and out is not too extreme, Clamp your camera to not exceed certain minimum and maximum values

```
//Clamp the Camera and set limits to avoid zooming in or out too much
TargetZoom = Mathf.Clamp(TargetZoom, 3, 6);
```

- Finally, to make the Zoom function smooth, use the Lerp function to move up or down from size A to size B

```
//Lerp function to smooth the transition from size 1 to size 2, AKA from current Orthographic Size to Target Orthographic Size
Cam.orthographicSize = Mathf.Lerp(Cam.orthographicSize, TargetZoom, Time.deltaTime * ZoomSpeed);
```

Particle Effects

When it comes to creating a particle effect for your game, there are two ways you can do it:

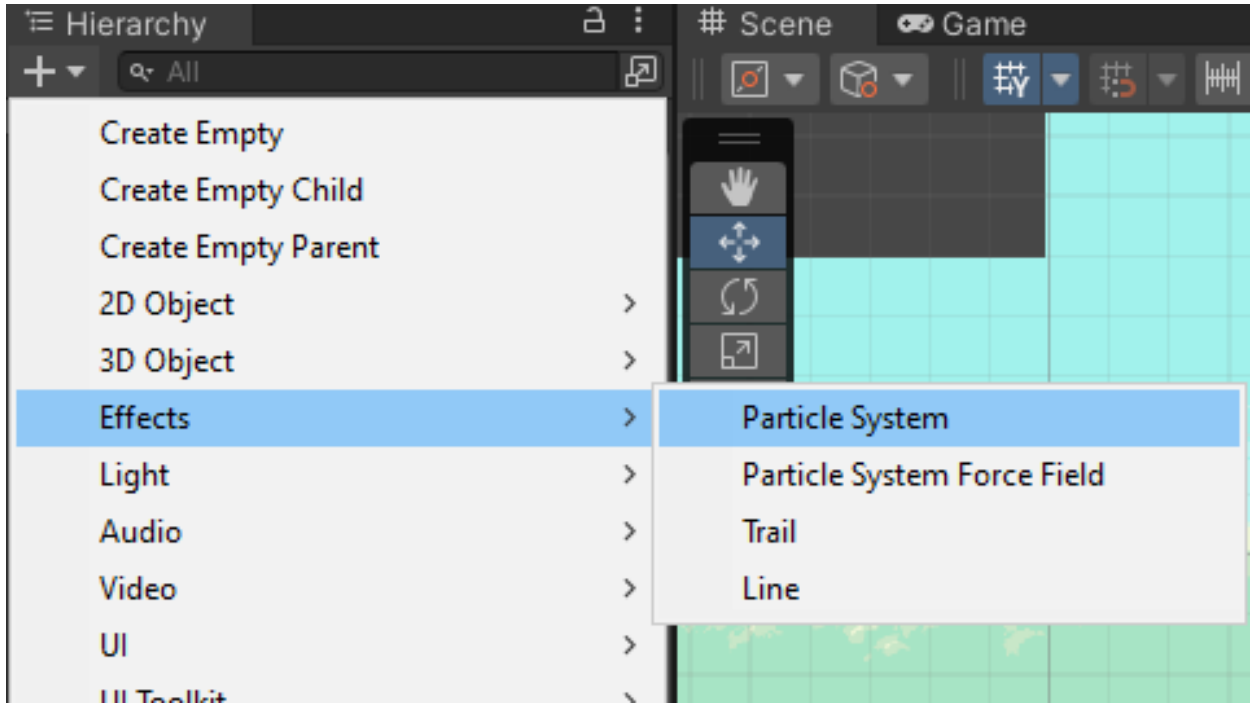
- Use a sprite sheet (aka animate it)

- **Use Unity's built-in Particle System**

This week, we'll be focusing on the Particle System

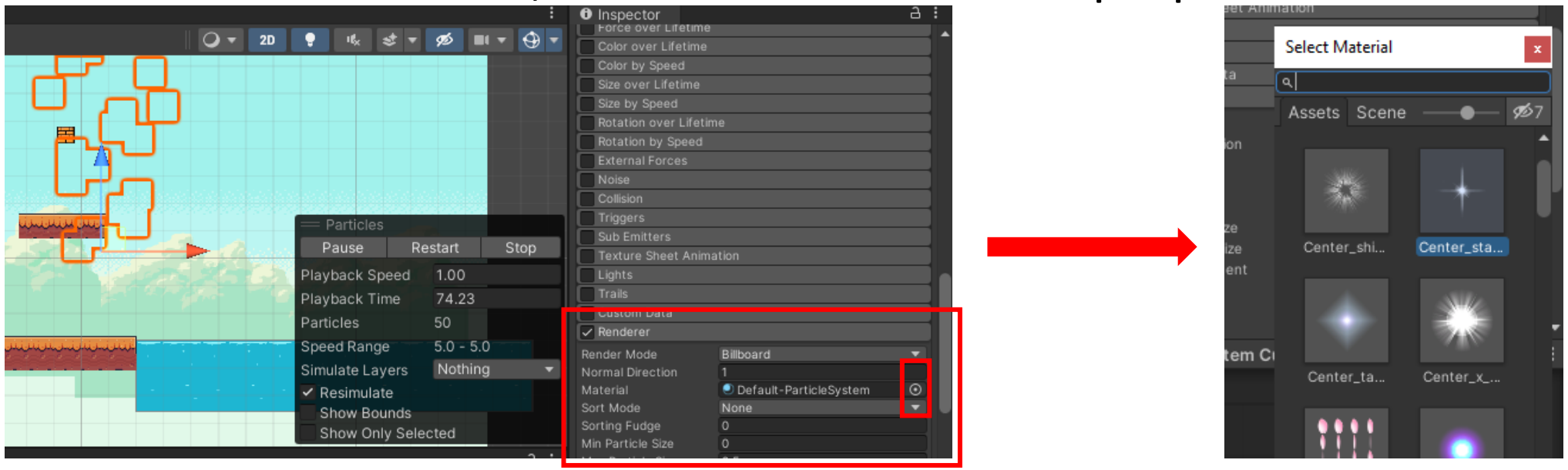
Exercise #14 – Particle Effect (Built-in System)

- To create a new particle system, click the + sign, choose Effects then Particle System.



Exercise #14 – Particle Effect (Built-in System) (Cont.)

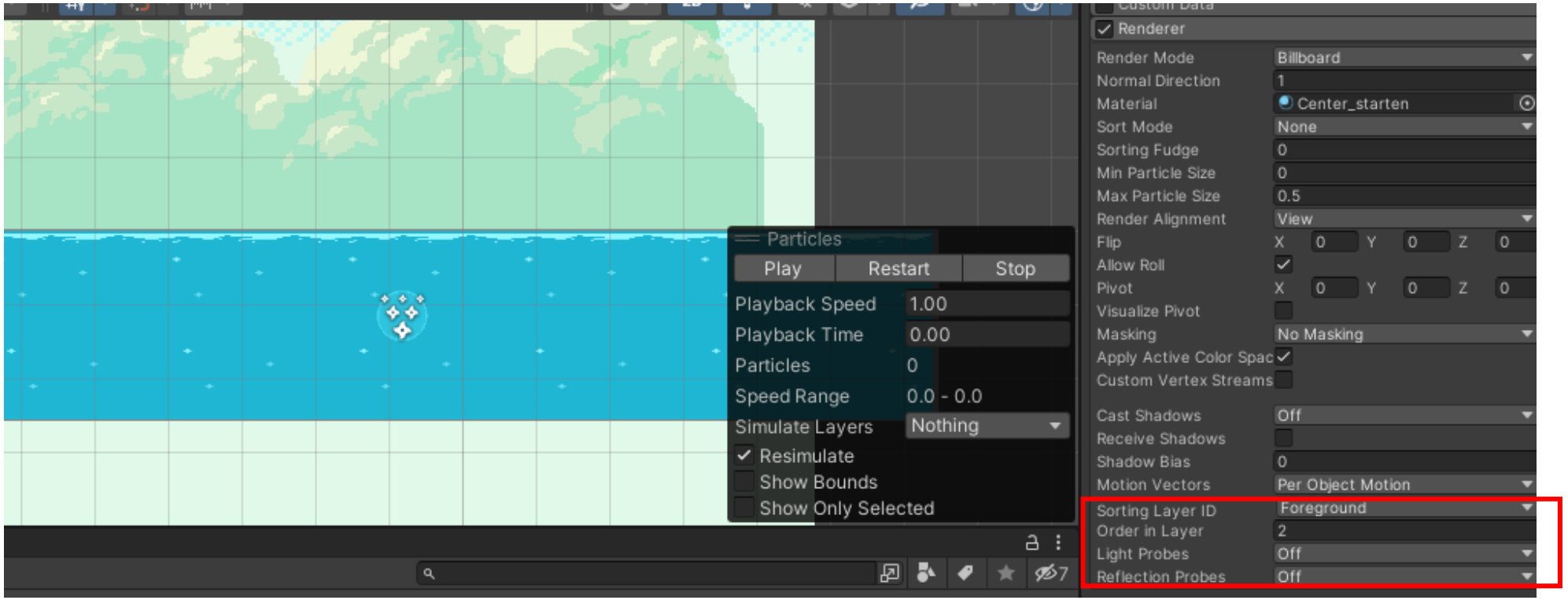
- Go the Inspector window, and scroll down to the Renderer checkbox. Make sure it's checked, then click it to view its properties:



- Click the circle in the Materials variable, choose one that looks like glitter or stars from the pop-up window.

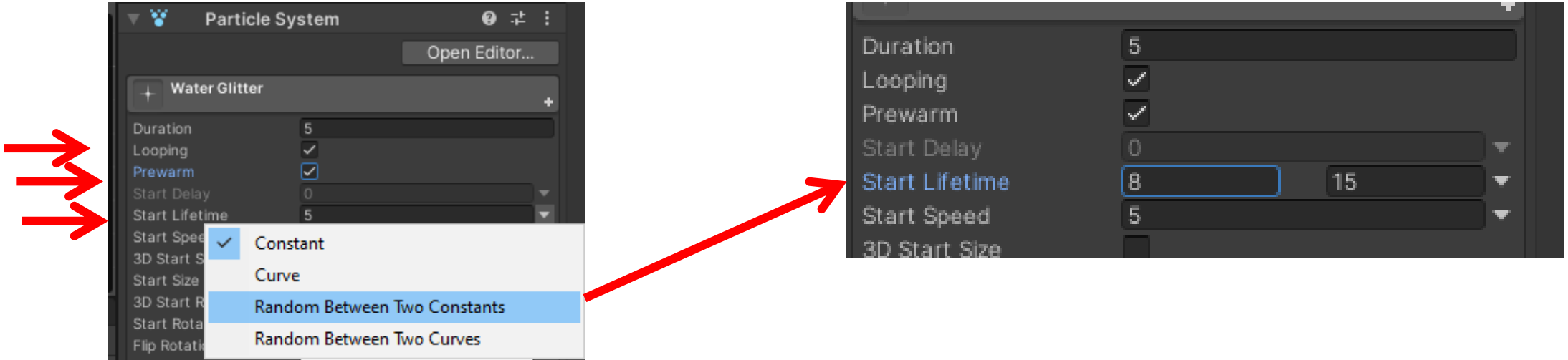
Exercise #14 – Particle Effect (Built-in System) (Cont.)

- Make sure the sorting layer is correct. In this case, we want a sparkling effect on the water, so it has to be on the Foreground.



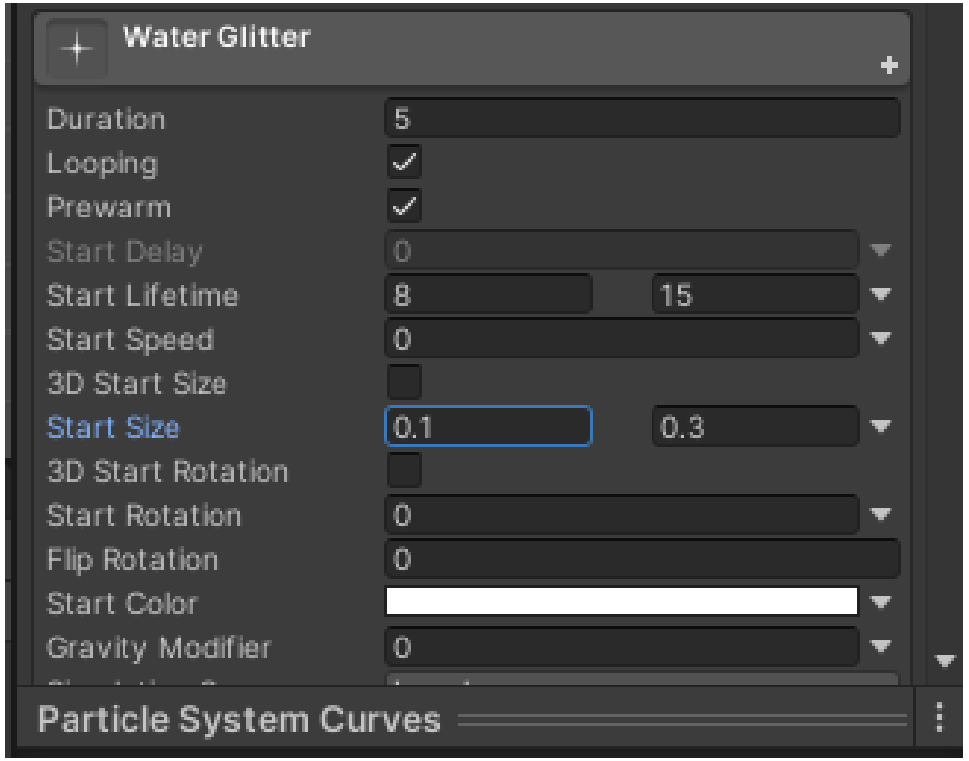
Exercise #14 – Particle Effect (Built-in System) (Cont.)

- Other settings you need to add:
- Make sure looping and Prewarm options in particle system are **checked**
- Go to the Start Lifetime option, and click the arrow to choose ‘Random Between Two Constants’
- Now put in two values in the **Start Lifetime**



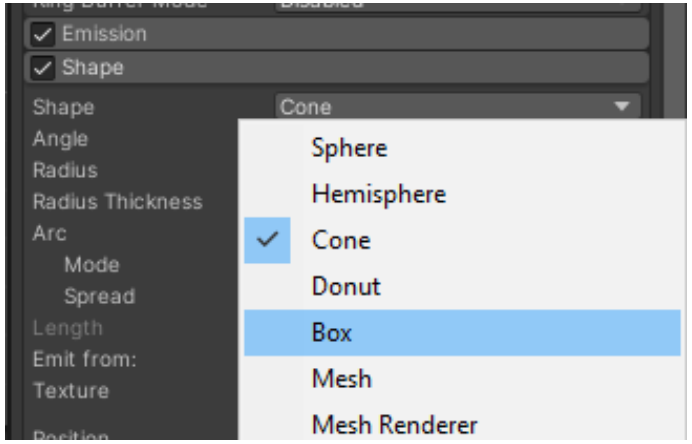
Exercise #14 – Particle Effect (Built-in System) (Cont.)

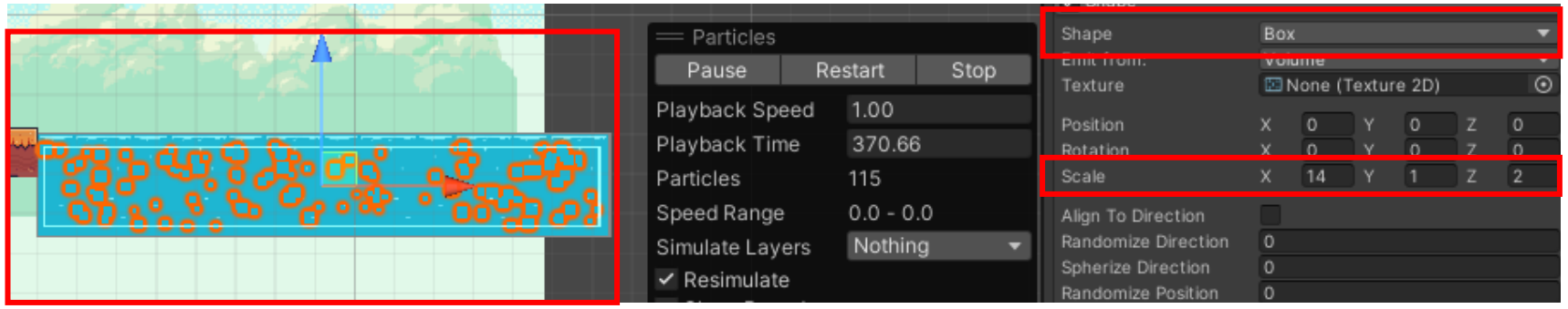
- ‘Start Speed’ variable at 0
- ‘Start Size’ can be also between two constants. Make it between 0.1 and 0.3, for example



Exercise #14 – Particle Effect (Built-in System) (Cont.)

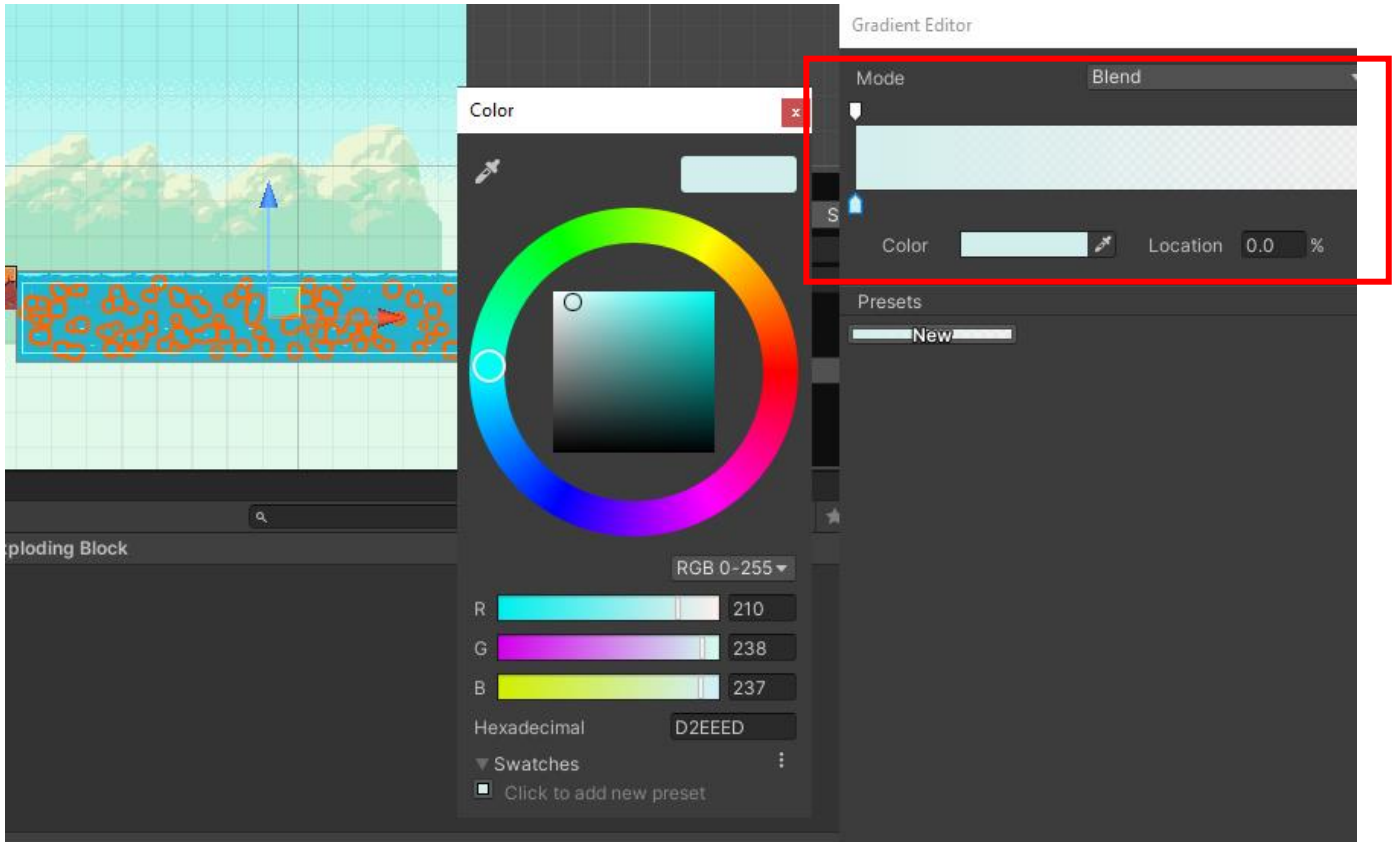
- Make sure the Shape option is checked, and change the shape to Box
- Change the scale value so it fits *within t*he limits of the lake





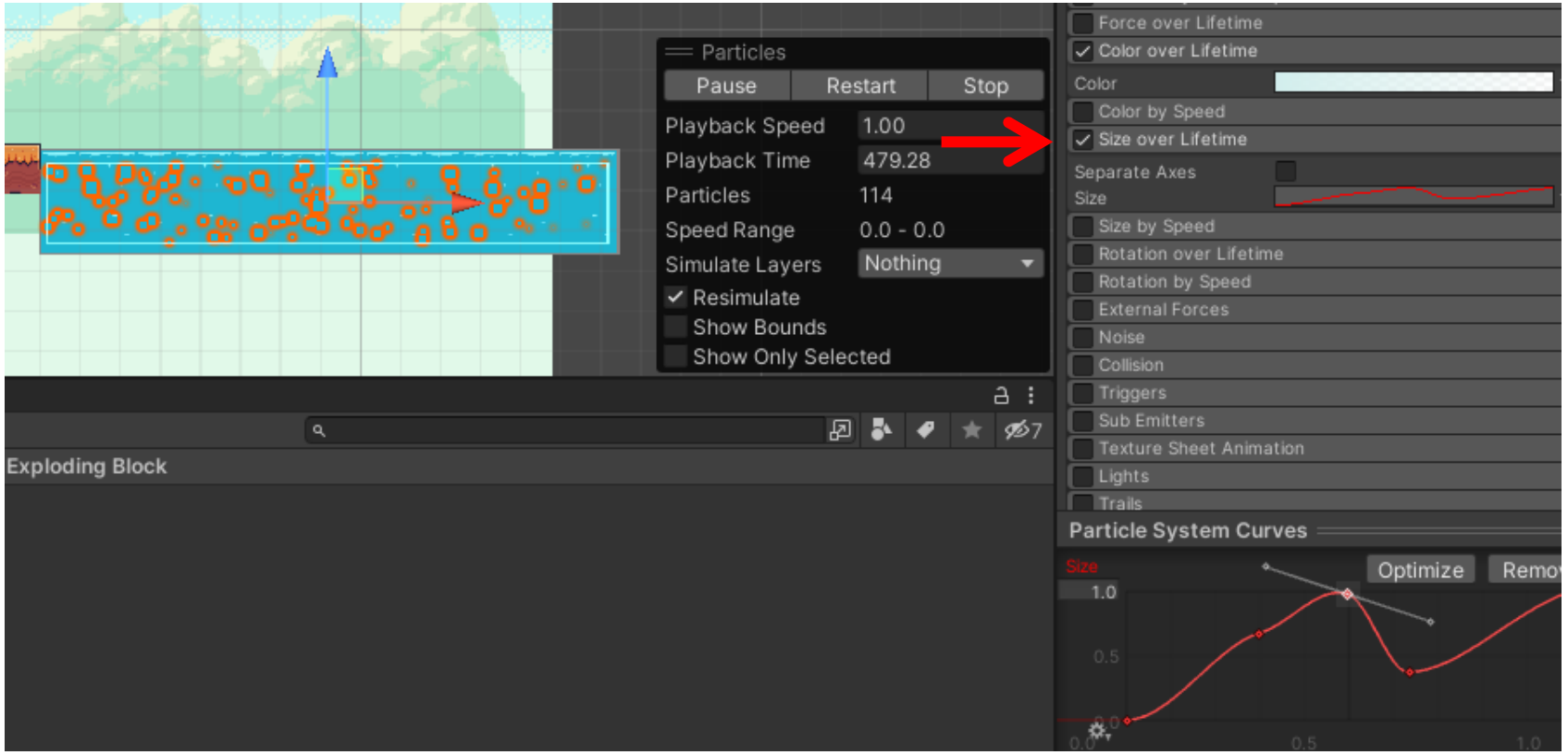
Exercise #14 – Particle Effect (Built-in System) (Cont.)

- Check the ‘Color Over Lifetime’ option, and change the coloring to be a very light blue. You can also lower the opacity at the corners if needed.



Exercise #14 – Particle Effect (Built-in System) (Cont.)

- You can also check the '**Size over Lifetime**' option, and play with the graph as it'll allow you to make individual particles change size and be more varied



Exercise #14 – Particle Effect (Built-in System) (Cont.)

- Make sure the effects icon on the right hand side of the Scene window is toggled, so you're able to see the effect in real time within the Scene view, not just the Game view

